AI-Driven 雙因子量化交易與回測系統

[>] 結合 LSTM 深度學習、PostgreSQL 資料庫與 Streamlit 全端整合

本系統以三層式軟體架構（Three-Tier Architecture）為骨幹，整合資料工程、深度學習、資料庫設計與互動式前端，展現完整的全端量化交易平台開發能力。

LSTM | PostgreSQL | Streamlit | Three-Tier Architecture

系統概覽 — 打破傳統腳本式回測的限制

架構突破 (ARCHITECTURE BREAKTHROUGH)

[>] 傳統量化回測痛點：
單一 Python 腳本、依賴靜態 CSV 讀取、缺乏資料庫持久化、無視覺化前端介面，難以擴展。

[>] 本系統突破：
導入三層式架構（Three-Tier Architecture），將展示層、邏輯層與資料層嚴格分離，具備企業級系統雛形。

[>] 核心設計原則：
關注點分離 (Separation of Concerns)、模組化開發、Type Hinting、Google Style Docstrings。

技術棧與模組 (TECH STACK & MODULES)

[>] 全端技術棧：

Python 3.13 | PyTorch (LSTM) | PostgreSQL | Streamlit | Plotly | SQLAlchemy

```
# 7 個核心模組 (Core Modules) ├── app.py # 前端儀表板 ├── model_core.py # AI 模型與特徵 ├── backtest_core.py # 回測引擎 ├── data_loader.py # ETL 資料載入 ├── database.py # 資料庫連線 ├── daily_trader.py # 自動化排程 └── unit_test.py # 單元測試
```

系統整合架構 — 三層式全端設計

層次 (LAYER)	技術棧 (TECHNOLOGY)	職責 (RESPONSIBILITIES)
展示層 (Presentation)	`Streamlit + Plotly`	提供 RWD 互動式網頁介面、動態圖表渲染、參數控制面板
邏輯層 (Business Logic)	`PyTorch (LSTM) + Pandas + NumPy`	執行特徵工程、AI 深度學習預測推論、Walk-forward 回測引擎
資料層 (Data Access)	`PostgreSQL + SQLAlchemy + yfinance`	外部 API 介接、資料清洗 ETL、持久化儲存、ACID 交易保護

資料流向與模組解耦 (DATA FLOW & DECOUPLING)

Yahoo API > ETL 清洗 > PostgreSQL > 特徵工程 > LSTM 模型 > 回測引擎 > Streamlit 前端

[>] **介面化通訊：**各模組嚴格透過函式介面（Function Interface）互相呼叫傳遞資料，不直接存取彼此的內部狀態，確保系統高內聚、低耦合。

程式設計開發 — 工程規範與模組化設計

測試驅動開發 (TDD)

[>] **單元測試實作：**使用 pytest 撰寫 unit_test.py，確保核心財務指標計算之絕對準確。

[>] **正向測試：**test_calculate_metrics_positive_return() 驗證正報酬情境下夏普值、勝率、最大回撤計算正確性。

[>] **反向測試 (防呆)：**test_calculate_metrics_zero_capital_error() 驗證本金為 0 時是否正確拋出 ValueError。

[>] **浮點數精度：**使用 pytest.approx() 容許極微小的浮點數誤差，確保跨平台數值比對穩定。

```
# unit_test.py 節錄 def test_calculate_metrics_positive_return():
... metrics = calculate_metrics(init_cap, final_cap, eq_curve,
trades) # 使用 pytest.approx 容許浮點數誤差 assert metrics["Total
Return (%)"] == pytest.approx(20.0) assert metrics["Win Rate (%)"]
== pytest.approx(66.67, abs=0.01)
```

型別提示與文件 (TYPE HINTING & DOCS)

[>] 全面導入 Python Type Hinting (Union, List, Tuple, Dict) 與 Google Style Docstrings，提升程式碼可讀性與 IDE 靜態檢查能力。

防呆機制 (DEFENSIVE PROGRAMMING)

[>] **型別轉換：**強制將 NumPy scalar 轉換為 Python 原生型別 (.item())，確保 JSON 序列化相容性。

[>] **異常處理：**包含 NaN 處理、空資料庫回傳保護、餘額不足自動取消交易等實盤防護。

提早停止機制 (EARLY STOPPING)

[>] 在 LSTM 模型訓練過程中，當 epoch_loss < 0.0005 時自動中斷訓練，有效避免模型過擬合 (Overfitting)，節省運算資源。

模型選擇 — 為什麼選 LSTM？

時間序列與 LSTM 優勢

[>] 股價資料本質：

金融市場數據屬於高度雜訊的「時間序列 (Time Series)」，具備強烈的「前後依賴性 (Temporal Dependency)」。

[>] 突破傳統 RNN 限制：

傳統 RNN 在處理長序列時容易遭遇「梯度消失 (Vanishing Gradient)」問題。LSTM (Long Short-Term Memory) 透過獨特的閘門機制解決此痛點，有效記憶長期趨勢並遺忘無用雜訊。

遺忘門 (Forget Gate) | 輸入門 (Input Gate) | 輸出門 (Output Gate)

系統 LSTM 架構設計

[>] PyTorch 網路拓撲：

針對量化特徵量身打造的深度學習架構，兼顧模型容量與泛化能力。

```
# PyTorch LSTM Hyperparameters input_size = 4 // Return, Close, Volume, RSI hidden_size = 50 // 隱藏層神經元數量 num_layers = 2 // 堆疊兩層捕捉複雜非線性模式 dropout = 0.2 // 20% Dropout 正則化防過擬合 # Output Layer nn.Linear(hidden_size, 1) // 預測明日報酬率
```

[>] 預測目標：

模型最終輸出單一連續數值，代表對「明日報酬率 (Return)」的預測，作為後續雙因子策略的 AI 訊號輸入。

特徵工程與超參數調整

特徵選擇 (FEATURE SELECTION)

[>] **4 個精選輸入特徵：**涵蓋價格、動能與流動性，提供模型多維度市場資訊。

Return

日報酬率。捕捉短期動能，作為模型預測目標。

Close

收盤價。反映市場當前定價水準與趨勢。

Volume

成交量。衡量市場參與度與籌碼流動性。

RSI

14日相對強弱指標。識別超買(>70)與超賣(<30)。

[>] **時間窗口 (Look-back)：**設定為 **60 天**，以過去 60 個交易日的資料序列預測明日方向。

防護機制與訓練參數 (TRAINING SETUP)

[>] **Data Leakage 防護：**
為避免未來函數污染，`MinMaxScaler` 僅在訓練集（前 80%）上進行 `fit`，再對全集進行 `transform`。

```
train_size = int(len(features) * 0.8)
scaler.fit(features[:train_size]) scaled_data =
scaler.transform(features)
```

[>] **模型訓練超參數：**

[-] Optimizer: Adam `(lr = 0.001)`

[-] Loss Function: `nn.MSELoss()` (均方誤差)

[-] Batch Size: 32 | Epochs: 100

[-] **Early Stopping：**當 `epoch_loss < 0.0005` 時提前終止，防止模型過擬合 (Overfitting)。

雙因子進出場策略設計

買進條件 (BUY CONDITIONS) — 三重確認

[>] **AI 預測看漲：** LSTM 輸出明日上漲 (`final_signal = True`).

[>] **趨勢確認濾網：** 當日收盤價須高於 MA20，確保多頭。

[>] **爆量動能確認：** 成交量高於 20 日均量 (`Factor_Pass = True`).

[>] **設計邏輯：** 結合 AI 與技術指標，降低單一指標假訊號風險。

賣出條件 (SELL CONDITIONS) — 三種觸發

[>] **AI 訊號反轉：** LSTM 預測轉為看跌時平倉或停損。

[>] **停損觸發：** 持倉報酬率 ≤ `-stop_loss_pct` (預設 5%)。

[>] **停利觸發：** 持倉報酬率 ≥ `+take_profit_pct` (預設 15%)。

風控參數與回測機制 (RISK MANAGEMENT & BACKTESTING)

[>] **盈虧比：** 停利 15% / 停損 5%，建立 3:1 的風險報酬配置。

[>] **逐日回測：** 以 T-1 資訊決定 T 日交易，避免未來函數。

績效指標計算 — 數字怎麼算的

指標 (METRIC)	計算方式 (FORMULA)	意義 (SIGNIFICANCE)
總報酬率 (Total Return)	`(final_capital / initial_capital - 1) * 100`	衡量策略在回測期間內的整體獲利能力。
夏普值 (Sharpe Ratio)	`(mean_return / std_return) * √252`	衡量承受每單位風險所獲得的超額報酬 (年化)。
最大回撤 (Max Drawdown)	`min((equity - cummax) / cummax) * 100`	衡量資金曲線從最高點滑落至最低點的最大縮水幅度，為核心風控指標。
勝率 (Win Rate)	`mean(trade_profits > 0) * 100`	衡量所有平倉交易中，獲利交易的佔比。

程式防呆與數值穩定性設計 (DEFENSIVE PROGRAMMING)

[>] **避免除以零錯誤 (ZeroDivisionError)：** 計算夏普值時，標準差加上極小值 `std_ret + 1e-9`，防止當報酬率完全一致時發生除以零崩潰。

[>] **NaN 數值保護：** 若夏普值計算結果為 NaN，強制設定為 `0.0`，確保前端圖表渲染不中斷。

[>] **JSON 序列化相容性：** NumPy 計算結果預設為 `numpy.float64`，透過 `hasattr(val, 'item')` 強制轉換為 Python 原生 `float`，避免 API 傳輸或前端解析失敗。

[>] **本金驗證：** 若初始本金 ≤ 0，主動拋出 `ValueError`，阻斷無效運算。

資料庫設計 — PostgreSQL 架構

核心資料表設計 (DATABASE SCHEMA)

[>] **捨棄 CSV：**全面採用關聯式資料庫，確保資料一致性與完整性。

1. Securities (證券基本資料)

儲存股票代號 (ticker_symbol)、公司名稱與產業類別。

2. Market_Data (市場行情)

儲存歷史 OHLCV 報價，作為模型訓練與回測的基礎資料。

3. Users (虛擬帳戶)

管理使用者 ID 與當前可用資金餘額 (current_balance)。

4. Transactions (交易明細)

記錄所有買賣動作 (BUY/SELL)、成交價格與數量，用於精算庫存。

5. Model_Signals (AI 預測備查)

記錄每日模型預測數值與產生的交易訊號，供後續績效追蹤。

預存程序與 ACID 保護 (STORED PROCEDURE)

[>] **交易原子性：**將扣款與寫入明細封裝於資料庫層級的 ExecuteTrade 預存程序中。

[>] **防呆機制：**在資料庫層面直接阻擋餘額不足的買單與無庫存的賣單，杜絕超額買進與負庫存。

```
CALL ExecuteTrade( user_id, ticker, action, price, quantity
);
```

增量更新 ETL (INCREMENTAL LOAD)

[>] **智慧型資料載入：**先查詢資料庫內該檔股票的「最新日期」，若發現資料過期，才啟動爬蟲抓取缺漏區間。

[>] **防重複寫入：**利用 SQL 的 ON CONFLICT DO NOTHING 語法，確保即使重複執行也不會產生重複的報價紀錄。

[>] **效能優化：**大幅減少 API 呼叫次數與網路傳輸延遲，適合每日自動排程執行。

網頁前端 — Streamlit 互動式儀表板

儀表板四大區塊 (DASHBOARD COMPONENTS)

1. 側邊欄控制 (Sidebar)

提供使用者選擇股票代號、設定回測時間區間與初始本金，動態觸發回測引擎。

2. 核心績效 (KPI Metrics)

使用 `st.metric` 呈現總報酬、夏普值、最大回撤與勝率，提供一目了然的策略評估。

3. 互動式圖表 (Plotly)

繪製資金曲線 (Equity Curve)，並在圖表上精準標記買進 (綠) 與賣出 (紅) 點位，支援縮放與懸停提示。

4. 交易明細表 (Trade Log)

以表格形式列出每筆交易的日期、動作、價格與損益，方便進行逐筆覆盤。

效能優化 (PERFORMANCE OPTIMIZATION)

[>] **資料快取機制：**
使用 Streamlit 的 `@st.cache_data` 裝飾器快取資料庫查詢結果。當使用者調整參數但未更換股票時，避免重複對 PostgreSQL 發起高成本的 I/O 請求。

```
@st.cache_data def load_data_from_db(ticker): return
db.fetch_market_data(ticker)
```

資安防護 (SECURITY & INJECTION PREVENTION)

[>] **防範 SQL Injection：**
全面棄用字串拼接 (String Concatenation) 組裝 SQL 語法。

[>] **參數化查詢：**
透過 SQLAlchemy ORM 與參數化查詢 (Parameterized Queries) 處理所有來自前端的輸入，確保系統免受惡意注入攻擊。

系統整合亮點 — 自動化交易排程機器人

中央協調器 (THE ORCHESTRATOR)

[>] **無縫串接：** `daily_trader.py` 作為系統的進入點 (Entry Point)，負責將資料庫、模型、策略與交易執行等獨立模組完美串接。

[>] **全自動化排程：** 設計為可直接掛載於 Linux `crontab` 或雲端排程服務，實現每日收盤後自動執行，無需人工干預 (Zero Manual Intervention)。

每日自動化執行流程 (DAILY EXECUTION PIPELINE)

STEP 01

data_loader.py

增量更新 ETL

連線 Yahoo Finance API，抓取最新日K線資料，並寫入 PostgreSQL 完成資料庫同步。

STEP 02

model_core.py

AI 推論預測

讀取最新 60 天特徵序列，載入預訓練之 LSTM 模型權重，輸出明日漲跌機率訊號。

STEP 03

Strategy Logic

雙因子策略決策

結合 AI 訊號與 MA20/均量等技術指標，判斷是否滿足買進或賣出條件。

STEP 04

database.py

交易執行與紀錄

若觸發交易，呼叫資料庫 Stored Procedure 扣款並寫入 Transactions 明細，完成閉環。

總結 — 四大技術能力展現

1. 程式設計開發 (PROGRAMMING)

[>] **測試驅動開發 (TDD)：**導入 pytest 確保核心財務指標計算之絕對準確。

[>] **工程規範：**全面使用 Type Hinting 與 Google Style Docstrings，提升可維護性。

[>] **防呆機制：**嚴謹的例外處理 (Exception Handling) 與型別轉換，確保系統穩定運行。

2. 資料庫設計 (DATABASE)

[>] **關聯式架構：**採用 PostgreSQL 取代靜態 CSV，確保資料一致性與完整性。

[>] **ACID 交易保護：**透過 Stored Procedure 封裝交易邏輯，杜絕超額買進與負庫存。

[>] **增量更新 ETL：**智慧型爬蟲僅抓取缺漏區間，大幅優化效能與降低 API 負載。

3. 網頁前端 (FRONTEND)

[>] **互動式儀表板：**使用 Streamlit 打造全端 UI，提供流暢的使用者體驗。

[>] **動態視覺化：**整合 Plotly 繪製專業 K 線圖與資金曲線，資料呈現直觀清晰。

[>] **狀態管理：**善用 Session State 實現跨頁面資料傳遞與效能快取 (Caching)。

4. 系統整合與架構 (ARCHITECTURE)

[>] **三層式架構：**嚴格分離展示層、邏輯層與資料層，實現高內聚低耦合。

[>] **模組化設計：**各模組透過明確的函式介面通訊，具備高度可擴展性。

[>] **全自動化排程：**透過 daily_trader.py 串接所有模組，實現從抓資料到下單的無人值守流程。